Variational-Dropout Stochasticity for GRPO Fine-Tuning of Latent-Reasoning Language Models

(revised)

Wooil Jung

May 5, 2026

Abstract

Latent-reasoning language models such as COCONUT (Hao et al., 2024) replace discrete chain-of-thought tokens with continuous hidden states fed recurrently into the next reasoning step. Because the latent recurrence is deterministic given the input, naïve policy-gradient methods such as GRPO cannot generate diverse rollouts in latent space. We propose to use a small dropout rate $p \approx 0.1$ as the source of stochasticity during rollout, and to *replay the same dropout mask* during the policy update so that the latent trajectory observed at rollout time is reproduced exactly. Following Gal & Ghahramani (2016), we further *share the mask across all latent steps within one rollout*, so each rollout corresponds to a single posterior sample $\tilde{\theta}(\xi)$ from the variational dropout distribution. We show that, under this construction, dropout noise is exogenous, the per-mask advantage $A^{(k)} = r^{(k)} - \mu_r$ (without standard-deviation normalization, following Liu et al. 2024) yields an unbiased policy-gradient estimator on the dropout-marginalized policy, and replaying the mask provides common-random-numbers variance reduction over freshly resampling at update time. We also give an inductive argument that the surrogate gradient is well defined at every latent depth T. A reference implementation is provided in `coconut_variational.py` and `run_grpo_dropout_variational.py`.

1 Revision summary

This revision corrects two errors and one ambiguity in the previous draft:

- **Hole 1 (advantage normalization).** The original draft used the GRPO advantage $A^{(k)} = (r^{(k)} - \mu_r)/(\sigma_r + \delta)$ and claimed unbiasedness in Theorem 4.4. The proof handled only the mean baseline μ_r and silently ignored σ_r. Because σ_r is a nonlinear function of every $r^{(j)}$ in the group (including $r^{(k)}$ itself), the control-variate identity does not extend to dividing by it, and the estimator is biased. This is the same observation made independently by Liu et al. (2024) ("Dr. GRPO"). In this revision we drop the σ_r normalization, use $A^{(k)} = r^{(k)} - \mu_r$, and recover $(1 - 1/K)\,\nabla J(\theta)$ exactly.
- **Hole 2 (CRN baseline).** The previous Proposition 4.5 compared mask replay to an estimator that re-used the rollout's reward $A(\xi, y)$ but evaluated the score function on a freshly sampled mask ξ'. Because y was not generated under ξ', the resampling estimator is malformed: it is biased and the mean equality assumed in the proof fails. This revision replaces the baseline with a *fresh-rollout* estimator $\hat{g}_{\text{indep}} = A(\xi', y')\,\nabla \log \pi_\theta(y' \mid x, \xi')$ where (ξ', y') is drawn afresh at update time. The CRN argument then goes through.
- **MC-dropout (formerly ambiguous).** The previous draft drew independent per-step masks m_t, which is incompatible with the Gal–Ghahramani interpretation in which a single

posterior sample θ' is reused throughout a forward pass. The implementation in `coconut_variational.py` now shares one mask $m_t \equiv m$ across all latent recurrence steps via an RNG-restore mechanism. Section 5.4 of this revision relies on that shared-mask construction and gives the Bayesian model-average reading rigorously.

The well-definedness lemma, the smoothness assumption, and the practical discussion are unchanged in substance.

2 Introduction and motivation

Group Relative Policy Optimization (GRPO; Shao et al., 2024) has become a standard reinforcement-learning objective for fine-tuning reasoning LLMs on math benchmarks. Its appeal is that the per-prompt advantage is computed purely from a group of K rollouts and a verifier reward, with no learned critic. GRPO requires that the K rollouts *differ*, because the group-mean baseline,

$$A^{(k)} \;=\; r^{(k)} - \mu_r, \qquad \mu_r \;=\; \tfrac{1}{K}\sum_{k=1}^{K} r^{(k)}, \tag{1}$$

is identically zero whenever all rollouts agree. In a standard token-level LLM, diversity is supplied automatically by sampling from the next-token distribution.

COCONUT-style latent reasoning, by contrast, does not sample. Hidden states $h_t \in \mathbb{R}^d$ are fed back into the model as the embedding of the next "token" for $t = 1, \ldots, T$ until a stopping criterion fires; only after the latent phase ends does the model emit discrete answer tokens $y_1, \ldots, y_L$. Conditional on the prompt x, the latent phase is a *deterministic* function of the parameters θ. This breaks GRPO: all K rollouts share the same h_T and (for greedy decoding) the same y.

We propose to inject stochasticity by enabling dropout during rollout, with one dropout mask drawn per rollout and held constant across all latent recurrence steps within that rollout (variational dropout, Gal & Ghahramani 2016). This yields per-rollout latent trajectories that depend on a Bernoulli mask $\xi \sim p(\xi)$ which is independent of θ, so GRPO can grade rollouts relative to one another. At update time we re-mount the *same* mask so that the gradient is computed against the trajectory that was actually graded.

Note on advantage normalization. We deliberately omit the standard-deviation normalization $1/(\sigma_r + \delta)$ that appears in the original GRPO formulation. As argued by Liu et al. (2024) and discussed in Section 5, dividing by a within-group statistic of the rewards introduces bias because σ_r depends on every individual $r^{(k)}$. Empirically, the normalization can be useful as an adaptive learning-rate trick, but it is not required for correctness and is omitted here for theoretical clarity.

3 Preliminaries

3.1 Coconut recurrence with variational dropout

Let $f_\theta : \mathbb{R}^d \to \mathbb{R}^d$ denote one Transformer step of the latent-reasoning model with parameters $\theta \in \mathbb{R}^D$. Standard COCONUT unrolls

$$h_t \;=\; f_\theta(h_{t-1}), \qquad h_0 \;=\; \mathrm{embed}_\theta(x), \qquad t = 1, \ldots, T. \tag{2}$$

With dropout at rate p and keep-probability $q := 1 - p$, draw a *single* Bernoulli mask $m \in \{0, 1\}^D$ with $m \sim \mathrm{Bern}(q)^{\otimes D}$ per rollout, and define rescaled parameters

$$\tilde{\theta}(\xi) \;:=\; \theta \odot m/q, \qquad \mathbb{E}_\xi\big[m/q\big] = \mathbf{1}. \tag{3}$$

The dropout-perturbed recurrence reuses the *same* $\tilde{\theta}(\xi)$ at every step:

$$h_t \;=\; f_{\tilde{\theta}(\xi)}(h_{t-1}), \qquad \xi \sim p(\xi), \qquad t = 1, \ldots, T. \tag{4}$$

This is variational dropout in the sense of Gal & Ghahramani (2016): one posterior sample $\tilde{\theta}(\xi)$ is reused across the entire recurrence, not redrawn per step. Once the latent phase finishes, answer tokens are sampled autoregressively,

$$y_\ell \;\sim\; \pi_\theta(\cdot \mid x, h_T, y_{<\ell}), \qquad \ell = 1, \ldots, L, \tag{5}$$

optionally with dropout enabled here as well; we absorb any such sampling randomness into ξ by reparameterizing the categorical samples with Gumbel noise when convenient. A scalar reward $r(y, x) \in [0, 1]$ is provided by an external verifier (e.g. exact-match on GSM8K).

Remark 1 (Implementation). The shared mask is realized in `coconut_variational.py` by capturing the CUDA + CPU RNG state at the start of each chain and restoring it at the start of each latent recurrence step (`_forward_base` branch on `inputs_embeds.shape[1] == 1`). Constant-shape dropout layers (residual / MLP / embedding) therefore draw bit-identical Bernoullis at every step. Variable-shape attention dropout, whose mask shape grows with kv-cache length, is checked by an auditor that compares the prefix overlap; in PyTorch's Philox-based dropout kernel the prefix is determined by the seed alone, so prefix masks at steps t and $t-1$ remain bit-identical on the overlap. See the file header for caveats on architectures (Qwen2 / LLaMA / Mistral / Gemma) that ship without residual / MLP dropout.

3.2 Augmented policy

The induced distribution over (ξ, y) given x is

$$\Pi_\theta(\xi, y \mid x) \;=\; p(\xi)\, \pi_\theta(y \mid x, \xi), \qquad \pi_\theta(y \mid x, \xi) := \prod_{\ell=1}^{L} \pi_\theta\big(y_\ell \,\big|\, x, h_T(\theta, \xi, x), y_{<\ell}\big). \tag{6}$$

We treat Π_θ as the policy that GRPO optimizes; the dropout mask is part of the action. The marginal policy

$$\bar{\pi}_\theta(y \mid x) \;:=\; \mathbb{E}_{\xi \sim p}\big[\pi_\theta(y \mid x, \xi)\big] \tag{7}$$

is the object whose expected reward we ultimately wish to maximize.

3.3 GRPO objective

For a group of K rollouts $\{(\xi^{(k)}, y^{(k)})\}_{k=1}^{K}$ generated under parameters θ_{old}, with rewards $r^{(k)} = r(y^{(k)}, x)$, define the advantage $A^{(k)}$ via (1) and the per-token importance ratio

$$\rho_\theta^{(k)}(\xi, y) \;:=\; \frac{\pi_\theta(y \mid x, \xi)}{\pi_{\theta_{\text{old}}}(y \mid x, \xi)}. \tag{8}$$

The clipped GRPO surrogate is

$$\mathcal{L}^{\text{GRPO}}(\theta) \;=\; \mathbb{E}_{x \sim \mathcal{D}}\, \frac{1}{K} \sum_{k=1}^{K} \min\Big(\rho_\theta^{(k)}\, A^{(k)},\; \text{clip}(\rho_\theta^{(k)},\, 1-\varepsilon,\, 1+\varepsilon)\, A^{(k)}\Big) \;-\; \beta\, \text{KL}\big(\pi_\theta \,\big\|\, \pi_{\text{ref}}\big). \tag{9}$$

4 Method: dropout as exogenous noise with mask replay

The proposed procedure is:

1. For each prompt x, draw K independent dropout masks $\xi^{(1)}, \ldots, \xi^{(K)} \sim p(\xi)$. Each mask is a single sample reused across all T latent steps within its rollout.
2. For each k, run the latent-reasoning recurrence (4) with mask $\xi^{(k)}$ (RNG seed stored on device), decode answer tokens $y^{(k)}$, compute reward $r^{(k)}$ and advantage $A^{(k)}$ via (1).
3. During the policy update, recompute $\pi_\theta(y^{(k)} \mid x, \xi^{(k)})$ with the *same* stored mask $\xi^{(k)}$, so that h_T at update time matches h_T at rollout time exactly when $\theta = \theta_{\text{old}}$. Form the GRPO loss (9) and step θ.

The construction rests on three observations.

(i) Independence of ξ from θ. The dropout mask ξ is sampled from a fixed distribution $p(\xi)$ that does not depend on θ. Hence, in (6), $\nabla_\theta \log p(\xi) = 0$, and

$$\nabla_\theta \log \Pi_\theta(\xi, y \mid x) \;=\; \nabla_\theta \log \pi_\theta(y \mid x, \xi). \tag{10}$$

The gradient signal therefore comes entirely from the ξ-conditional log likelihood of the answer tokens, which is differentiable through the unrolled latent path.

(ii) Stochastic-computation-graph view. With ξ fixed, $h_T = h_T(\theta, \xi, x)$ is a deterministic, almost everywhere differentiable function of θ. The estimator $A^{(k)} \nabla_\theta \log \pi_\theta(y^{(k)} \mid x, \xi^{(k)})$ is a score-function estimator on Π_θ with the dropout draw treated as an exogenous input (Schulman et al. 2015, "Gradient estimation using stochastic computation graphs"). Because y is categorical we cannot apply the pathwise / reparameterization trick at the answer-token boundary; the gradient through the latent dynamics nonetheless flows by ordinary backpropagation through the unrolled recurrence.

(iii) Mask replay = common random numbers. At update time we evaluate π_θ on the same $\xi^{(k)}$ that was used at rollout time. This couples the rollout-time and update-time trajectories so that the only source of difference between them is the parameter update $\theta - \theta_{\text{old}}$ itself. At $\theta = \theta_{\text{old}}$ the importance ratio $\rho^{(k)}_{\theta_{\text{old}}}$ is identically 1 and h_T at update time exactly reproduces h_T at rollout time — a discrepancy that would otherwise contribute additional variance to the estimator. Section 5.3 formalizes this CRN claim against a fresh-rollout baseline.

5 Theoretical analysis

5.1 Well-definedness of the surrogate gradient (induction on T)

Assumption 2. For every fixed mask ξ and almost every $\theta \in \mathbb{R}^D$ in Lebesgue measure, the map $\theta \mapsto f_{\tilde{\theta}(\xi)}(h)$ is differentiable in θ for almost every h, and the answer-token log-likelihood $\log \pi_\theta(y \mid x, h_T, y_{<\ell})$ is differentiable in both θ and h_T.

This is satisfied by every standard Transformer block: the dropout rescaling $\theta \odot m/q$ is linear in θ, and the remaining nonlinearities (LayerNorm, GELU, softmax) are almost everywhere differentiable.

Lemma 3 (Well-defined latent gradient at depth T)**.** *Under Assumption 2, for every $T \geq 0$ and every ξ, the Jacobian $\partial h_T/\partial\theta$ exists almost everywhere in θ and is given by the recursion*

$$\frac{\partial h_t}{\partial \theta} \;=\; \frac{\partial f_{\tilde{\theta}}}{\partial \theta}\Big|_{h_{t-1}} \cdot \frac{\partial \tilde{\theta}}{\partial \theta} \;+\; \frac{\partial f_{\tilde{\theta}}}{\partial h}\Big|_{h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial \theta}, \qquad \frac{\partial h_0}{\partial \theta} \;=\; \frac{\partial\, \mathrm{embed}_\theta(x)}{\partial \theta}, \tag{11}$$

where $\partial\tilde{\theta}/\partial\theta = \mathrm{diag}(m)/q$ is the dropout-rescaled identity. Consequently $\nabla_\theta \log \pi_\theta(y \mid x, \xi)$ is well defined.

Proof. By induction on t.

Base case ($t = 0$). $h_0 = \mathrm{embed}_\theta(x)$ is differentiable in θ by assumption, giving the stated initial Jacobian.

Inductive step. Assume $\partial h_{t-1}/\partial\theta$ exists. Write $h_t = f(\tilde{\theta}, h_{t-1})$, viewing f as a function of two arguments. By the multivariate chain rule and Assumption 2,

$$\frac{\partial h_t}{\partial \theta} \;=\; \frac{\partial f}{\partial \tilde{\theta}} \cdot \frac{\partial \tilde{\theta}}{\partial \theta} \;+\; \frac{\partial f}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial \theta},$$

and $\partial\tilde{\theta}/\partial\theta = \mathrm{diag}(m)/q$ from (3). Both terms exist (the second by the inductive hypothesis), giving (11).

Finally, since $\log \pi_\theta(y \mid x, h_T, y_{<\ell})$ is differentiable in both θ and h_T, one more chain-rule application yields $\nabla_\theta \log \pi_\theta(y \mid x, \xi) = \sum_\ell [\partial_\theta \log \pi_\theta]_{\text{direct}} + [\partial_{h_T} \log \pi_\theta] \cdot \partial h_T/\partial\theta$. □

Remark 4. The recursion (11) is what `loss.backward()` implements when the dropout mask is held constant via RNG restore: PyTorch's `torch.nn.functional.dropout` treats the rescaled mask as a constant during backprop, which is exactly $\partial\tilde{\theta}/\partial\theta = \mathrm{diag}(m)/q$.

5.2 Unbiasedness of the dropout-GRPO surrogate

Throughout this subsection we condition on a single prompt x and treat the K rollouts as i.i.d. draws from $\Pi_{\theta_{\text{old}}}(\cdot \mid x) = p(\xi)\, \pi_{\theta_{\text{old}}}(y \mid x, \xi)$.

Theorem 5 (Unbiased policy-surrogate gradient)**.** *Let $J(\theta) := \mathbb{E}_{x\sim\mathcal{D}}\, \mathbb{E}_{(\xi,y)\sim\Pi_\theta(\cdot|x)}[r(y,x)]$ be the expected reward under the augmented policy* (6)*. Define the surrogate part of the GRPO loss (excluding the KL term) as $\mathcal{L}^{\text{surr}}(\theta) := \mathcal{L}^{\text{GRPO}}(\theta) + \beta\, \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$. Under the advantage definition $A^{(k)} = r^{(k)} - \mu_r$ in* (1)*,*

$$\nabla_\theta \mathcal{L}^{\text{surr}}(\theta)\Big|_{\theta=\theta_{\text{old}}} \;=\; \Big(1 - \tfrac{1}{K}\Big)\, \nabla_\theta J(\theta)\Big|_{\theta_{\text{old}}}. \tag{12}$$

Equivalently, $\nabla_\theta \mathcal{L}^{\text{GRPO}}(\theta)\big|_{\theta_{\text{old}}} = (1 - 1/K)\, \nabla J(\theta_{\text{old}}) - \beta\, \nabla_\theta \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})\big|_{\theta_{\text{old}}}$.

Proof. At $\theta = \theta_{\text{old}}$ we have $\rho^{(k)}_{\theta_{\text{old}}} \equiv 1$, so the clip is inactive and the gradient of the min reduces to the gradient of the unclipped ratio:

$$\nabla_\theta \rho^{(k)}_\theta\big|_{\theta_{\text{old}}} \;=\; \nabla_\theta \log \pi_\theta(y^{(k)} \mid x, \xi^{(k)})\big|_{\theta_{\text{old}}}.$$

Therefore, with $s^{(k)} := \nabla_\theta \log \pi_\theta(y^{(k)} \mid x, \xi^{(k)})$ evaluated at θ_{old},

$$\nabla_\theta \mathcal{L}^{\text{surr}}\big|_{\theta_{\text{old}}} \;=\; \mathbb{E}_{x,\, \{(\xi^{(k)}, y^{(k)})\}} \left[\frac{1}{K} \sum_{k=1}^{K} (r^{(k)} - \mu_r)\, s^{(k)} \right]. \tag{13}$$

We expand the baseline term. Because rollouts are i.i.d. given x,

$$\mathbb{E}[\mu_r\, s^{(k)}] \;=\; \frac{1}{K}\sum_{j=1}^{K}\mathbb{E}[r^{(j)}\, s^{(k)}] \;=\; \frac{1}{K}\mathbb{E}[r^{(k)}\, s^{(k)}] \;+\; \frac{1}{K}\sum_{j\neq k}\mathbb{E}[r^{(j)}]\,\mathbb{E}[s^{(k)}].$$

The score function has zero mean: $\mathbb{E}[s^{(k)}] = \mathbb{E}_{\xi}\mathbb{E}_{y|\xi}[\nabla \log \pi_\theta(y \mid x, \xi)] = \nabla \mathbb{E}_y[1] = 0$. Hence

$$\mathbb{E}[\mu_r\, s^{(k)}] \;=\; \tfrac{1}{K}\mathbb{E}[r^{(k)}\, s^{(k)}],$$

and substituting back,

$$\begin{aligned}\mathbb{E}[(r^{(k)} - \mu_r)\, s^{(k)}] &= \mathbb{E}[r^{(k)}\, s^{(k)}] - \tfrac{1}{K}\mathbb{E}[r^{(k)}\, s^{(k)}] \\ &= \big(1 - \tfrac{1}{K}\big)\,\mathbb{E}[r^{(k)}\, s^{(k)}].\end{aligned}$$

By (10), $s^{(k)} = \nabla_\theta \log \Pi_\theta(\xi^{(k)}, y^{(k)} \mid x)$, so $\mathbb{E}[r^{(k)}\, s^{(k)}] = \nabla_\theta J(\theta)|_{\theta_{\text{old}}}$ is the standard REINFORCE policy gradient on the augmented policy. Averaging over k preserves this and yields (12).

The KL term contributes $-\beta\, \nabla_\theta \mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})|_{\theta_{\text{old}}}$, which is in general nonzero (it vanishes only if $\pi_{\text{ref}} = \pi_{\theta_{\text{old}}}$, e.g. if the reference is reset each step). This term is the regularizer's contribution and has known gradient. □

Corollary 6 (Optimization on the marginal policy)**.** *The estimator targets $\nabla J(\theta)$, the expected reward of the mask-marginalized policy* (7)*, since $\mathbb{E}_\xi[\pi_\theta(y \mid x, \xi)] = \bar{\pi}_\theta(y \mid x)$ implies $\mathbb{E}_{\xi, y\sim\Pi_\theta}[r] = \mathbb{E}_{y\sim\bar{\pi}_\theta}[r]$. In other words, optimizing Π_θ is exactly optimizing expected reward under the dropout ensemble.*

Remark 7 (Why σ_r is dropped). If we instead used $A^{(k)} = (r^{(k)} - \mu_r)/(\sigma_r + \delta)$, the calculation in the proof above would require expanding $\mathbb{E}[(r^{(k)} - \mu_r)/(\sigma_r + \delta) \cdot s^{(k)}]$. Because σ_r is a nonlinear function of every $r^{(j)}$ (including $r^{(k)}$), the factorization $\mathbb{E}[r^{(j)}\, s^{(k)}] = \mathbb{E}[r^{(j)}]\mathbb{E}[s^{(k)}]$ used for $j \neq k$ breaks: σ_r couples $s^{(k)}$ to all rewards through a non-affine function. The resulting estimator overweights low-variance prompts (where $1/\sigma_r$ is large) and underweights high-variance ones, distorting the per-prompt gradient direction. Liu et al. (2024) report this bias empirically and trace prompt-difficulty drift to it. We therefore drop the σ_r normalization throughout, at the cost of losing automatic adaptive scaling — a tradeoff that recovers theoretical correctness in exchange for one extra learning-rate hyperparameter.

5.3 Variance reduction via mask replay (CRN)

We compare two unbiased estimators of $\nabla_\theta J$ at parameter $\theta = \theta_{\text{old}} + \Delta$ for small Δ.

- **Mask-replay (CRN) estimator.** The same ξ used at rollout time is reused at update time. Conditional on x and on the rollout draw $(\xi, y) \sim \Pi_{\theta_{\text{old}}}(\cdot \mid x)$,

$$\hat{g}_{\text{CRN}}(\theta; \xi, y) \;:=\; A(\xi, y)\, \nabla_\theta \log \pi_\theta(y \mid x, \xi). \tag{14}$$

- **Fresh-rollout estimator.** At update time we draw a new mask ξ' and sample $y' \sim \pi_{\theta_{\text{old}}}(\cdot \mid x, \xi')$ from a fresh rollout, recompute its advantage $A(\xi', y')$ against an independent group, and form

$$\hat{g}_{\text{fresh}}(\theta; \xi', y') \;:=\; A(\xi', y')\, \nabla_\theta \log \pi_\theta(y' \mid x, \xi'). \tag{15}$$

Remark 8 (On the previous draft's baseline). The original Proposition 4.5 used $A(\xi, y)\,\nabla_\theta \log \pi_\theta(y \mid x, \xi')$ with $\xi' \perp \xi$ fresh and y from the rollout. This estimator is biased — y was not sampled under ξ', so the score-function identity does not hold — and a CRN argument relative to it is meaningless. The fresh-rollout baseline above is the correct counterfactual: "what if you had not replayed the mask, but instead drawn a brand-new rollout at update time."

Proposition 9 (CRN variance reduction)**.** *Both $\hat{g}_{\mathrm{CRN}}$ and $\hat{g}_{\mathrm{fresh}}$ are unbiased estimators of $\nabla J(\theta)$ at $\theta = \theta_{\mathrm{old}}$ (modulo the $(1-1/K)$ scale of Theorem 5). Suppose the function $\theta \mapsto A(\xi, y)\,\nabla_\theta \log \pi_\theta(y \mid x, \xi)$ is L-Lipschitz in θ for $\Pi_{\theta_{\mathrm{old}}}$-almost every (ξ, y), and that the difference*

$$\Delta(\theta; \xi, y) \;:=\; A(\xi, y)\,\nabla_\theta \log \pi_\theta(y \mid x, \xi) - A(\xi, y)\,\nabla_\theta \log \pi_{\theta_{\mathrm{old}}}(y \mid x, \xi)$$

satisfies $\|\Delta(\theta; \xi, y)\| \le L\|\theta - \theta_{\mathrm{old}}\|$. Then

$$\mathrm{Var}[\hat{g}_{\mathrm{CRN}}(\theta)] \;\le\; \mathrm{Var}[\hat{g}_{\mathrm{fresh}}(\theta)] \;+\; 2L\,\|\theta - \theta_{\mathrm{old}}\|\,\sqrt{\mathrm{Var}[\hat{g}_{\mathrm{CRN}}(\theta_{\mathrm{old}})]} \;+\; L^2\|\theta - \theta_{\mathrm{old}}\|^2. \tag{16}$$

In particular, in the trust-region regime $\|\theta - \theta_{\mathrm{old}}\| \to 0$ the CRN variance lower-bounds the fresh-rollout variance up to $O(\|\theta - \theta_{\mathrm{old}}\|)$ slack. At $\theta = \theta_{\mathrm{old}}$ exactly, $\hat{g}_{\mathrm{CRN}}(\theta_{\mathrm{old}})$ has the same per-sample value distribution as $\hat{g}_{\mathrm{fresh}}(\theta_{\mathrm{old}})$, but reuses (ξ, y) across calls, eliminating the across-call variance contribution from resampling.

Proof sketch. Write each estimator as a value at θ_{old} plus a perturbation: $\hat{g}_{\mathrm{CRN}}(\theta) = \hat{g}_{\mathrm{CRN}}(\theta_{\mathrm{old}}) + \Delta(\theta; \xi, y)$ and analogously for $\hat{g}_{\mathrm{fresh}}$ with (ξ', y'). The two zero-order terms have the same distribution because (ξ, y) and (ξ', y') are both drawn from $\Pi_{\theta_{\mathrm{old}}}(\cdot \mid x)$. The variance bound follows from Cauchy–Schwarz on the cross term and the Lipschitz hypothesis. This is the standard CRN argument (Glasserman & Yao, 1992, Thm 3.1) specialized to a score-function estimator coupled across θ values via shared exogenous noise. □

Remark 10 (Practical reading). The bound says: at θ_{old} both estimators have identical variance, and CRN's advantage shows up cumulatively across mini-epochs as θ drifts from θ_{old}. The fresh-rollout baseline pays an additional across-mask variance term that CRN avoids by construction. In a trust-region regime where $\|\theta - \theta_{\mathrm{old}}\|$ is kept small by the PPO clip, this advantage is the dominant variance reduction.

5.4 MC-dropout / Bayesian model-average reading

Following Gal & Ghahramani (2016), the shared-mask construction (4) is interpretable as a single posterior sample from a structured variational distribution $q_\theta(\theta')$ over parameters: $\tilde{\theta}(\xi) = \theta \odot m(\xi)/q$ with $m \sim \mathrm{Bern}(q)^{\otimes D}$. Crucially, because the mask is held constant across all T latent steps, the entire latent recurrence is evaluated at a single $\tilde{\theta}(\xi)$, matching the variational inference derivation. The marginal policy (7) is then exactly a Bayesian model-average policy:

$$\bar{\pi}_\theta(y \mid x) \;=\; \mathbb{E}_{\theta' \sim q_\theta}[\pi_{\theta'}(y \mid x)]. \tag{17}$$

Theorem 5 and Corollary 6 together state that dropout-GRPO performs ensemble-aware policy optimization: each gradient step is the Monte Carlo average of per-ensemble-member policy gradients, with the group-mean baseline acting as a control variate.

Remark 11 (Implementation caveats). The Bayesian reading holds cleanly for layers whose dropout mask shape does not depend on the step index t (residual / MLP / embedding dropout), because RNG restore reproduces the mask bit-identically. For attention dropout in modern decoders

(Qwen2 / LLaMA / Mistral / Gemma) the kv-cache length grows with t, so the mask shape varies; on the shape-overlap prefix the mask is still bit-identical (PyTorch Philox kernel property), but the suffix differs. This is a per-layer approximation of variational dropout; the proof framework in Sections 5.2–5.3 is unaffected because it only requires ξ to be exogenous and replayable. The `_DropoutMaskAuditor` class in `coconut_variational.py` verifies the bit-identity property at runtime.

6 Algorithm

Algorithm 1: Dropout-GRPO for latent-reasoning LLMs (revised).

Inputs: base policy $\pi_{\theta_{\text{old}}}$, prompts $\mathcal{D}$, group size K, dropout rate p, latent depth T, clip ε, KL coef β, mini-epochs E.

1. For each prompt $x \sim \mathcal{D}$:
   a. *Rollout* (train mode, dropout on). For $k = 1, \ldots, K$: sample fresh dropout RNG seed for $\xi^{(k)}$, store seed; capture chain RNG state; unroll latent recurrence (4) restoring the chain seed at every latent step (locks $m_t \equiv m$); decode $y^{(k)}$; record per-token logprobs $\log \pi_{\theta_{\text{old}}}(y^{(k)} \mid x, \xi^{(k)})$; compute $r^{(k)} = \text{verifier}(y^{(k)}, x)$.
   b. Compute advantages $A^{(k)} = r^{(k)} - \mu_r$ as in (1). *(No σ_r normalization.)*
   c. *Update* (mask replay). For mini-epoch $e = 1, \ldots, E$ and each k: regenerate $\xi^{(k)}$ from stored seed (RNG-restore at every latent step), recompute $\log \pi_\theta(y^{(k)} \mid x, \xi^{(k)})$, form $\rho^{(k)} = \pi_\theta / \pi_{\theta_{\text{old}}}$, take a gradient step on $\mathcal{L}^{\text{GRPO}}$ from (9).
   d. Set $\theta_{\text{old}} \leftarrow \theta$.

7 Practical concerns and open questions

Sensitivity of h_T to ξ. If dropout perturbations to h_T do not change the argmax of the answer-token distribution, all K rollouts agree, advantages collapse to zero, and no gradient is produced. This is the dropout-rate analogue of temperature in token-level RL. A diagnostic to monitor is the within-group standard deviation of $r^{(k)}$ (used in our implementation for zero-variance group filtering); if $\sigma_r \to 0$, raise p or fold token-sampling Gumbel noise into ξ. Note that σ_r is used here only as a *diagnostic / filter*, not in the advantage — consistent with Theorem 5.

Importance-ratio explosion. Under mask replay, $\rho_\theta(\xi, y)$ depends only on the parameter update $\theta - \theta_{\text{old}}$ (the mask-induced logit shifts are common to numerator and denominator), so the standard PPO Lipschitz argument applies directly. Without mask replay, ρ would also reflect mask resampling and could be large because dropping different units changes logits non-smoothly — this is the variance source that mask replay removes (cf. Proposition 9).

Validity of clip under CRN. The clip operator in (9) is non-smooth, so Theorem 5 as stated holds only at $\theta = \theta_{\text{old}}$ (initial step within each mini-epoch). Standard PPO analysis (Schulman et al., 2017, §5) extends the bound to small steps under a Lipschitz condition on ρ_θ, which under mask replay reduces to Lipschitzness of π_θ in θ at fixed ξ — a mild condition implied by Assumption 2 together with bounded weight matrices. Empirical monitoring of the maximum log-ratio is recommended.

KL anchor. Theorem 5 retains the KL term in the gradient ($-\beta\nabla\mathrm{KL}$ contribution). If one wishes the surrogate-only unbiasedness to be a statement about the full GRPO loss, set $\pi_{\text{ref}} \leftarrow \pi_{\theta_{\text{old}}}$ at the start of each update (reference renewal), so that $\nabla\mathrm{KL}|_{\theta_{\text{old}}} = 0$. Otherwise the KL gradient is a known regularizer term and Theorem 5 still pins the policy-gradient component exactly.

Train/deploy mismatch. Inference typically disables dropout, so the deployed policy is $\pi_\theta(y \mid x)$ with no mask, rather than the marginal $\bar{\pi}_\theta$. Two options: (a) keep dropout on at deploy time and average over masks, or (b) accept a bias whose magnitude depends on the network's per-layer Lipschitz constants; we do not give a closed-form bound here and recommend empirical comparison.

Memory for stored masks. Storing $\xi^{(k)}$ for K rollouts costs $\mathcal{O}(K \cdot \text{seed\_size})$ bits when implemented via per-chain RNG seeds, since the mask is regenerated deterministically at update time. This is the recommended implementation and the one used in `coconut_variational.py`.

Constant $(1 - 1/K)$ scaling. Theorem 5 introduces a $(1 - 1/K)$ multiplicative factor on the policy gradient relative to the true ∇J. For $K \geq 4$ this is ≥ 0.75 and is absorbed into the learning rate without practical consequence. For very small K (e.g. $K = 2$ giving factor 0.5) the effective learning rate halves; bear this in mind when sweeping group sizes.

8 Summary

- Latent-reasoning models break GRPO because the latent phase is deterministic; variational dropout (one mask per rollout, shared across all latent steps) supplies the missing stochasticity without changing the model class.
- Dropout noise is exogenous (independent of θ); the score function reduces to the ξ-conditional log likelihood (Eq. 10).
- The latent gradient $\partial h_T/\partial\theta$ is well defined at every depth T (Lemma 3), justifying backprop through the unrolled latent recurrence.
- GRPO with $A^{(k)} = r^{(k)} - \mu_r$ (no σ_r normalization) and replayed masks has surrogate gradient equal to $(1 - 1/K)\,\nabla J(\theta)$ at $\theta = \theta_{\text{old}}$ (Theorem 5, Corollary 6). The σ_r normalization is dropped because it introduces bias (Remark 7; cf. Liu et al. 2024).
- Replaying the rollout-time mask gives a CRN coupling that lower-bounds variance against a fresh-rollout estimator (Proposition 9), with the bound becoming tight in the trust-region regime.
- The shared-mask construction makes the marginal policy a true Bayesian model average $\bar{\pi}_\theta(y \mid x) = \mathbb{E}_{\theta' \sim q_\theta}[\pi_{\theta'}(y \mid x)]$ (Section 5.4, Eq. 17).
- Practical caveats are: h_T-sensitivity, importance-ratio control, clip-region Lipschitz behavior, KL anchor handling, and train/deploy mismatch.

References (informal)

- Hao, S. et al. (2024). *Training large language models to reason in a continuous latent space.* (Coconut.)

- Shao, Z. et al. (2024). *DeepSeekMath: Pushing the limits of mathematical reasoning in open language models.* (GRPO.)
- Liu, Z. et al. (2024). *Understanding R1-Zero-like training: A critical perspective.* (Dr. GRPO — bias of σ_r normalization.)
- Schulman, J. et al. (2017). *Proximal policy optimization algorithms.*
- Schulman, J. et al. (2015). *Gradient estimation using stochastic computation graphs.*
- Gal, Y. & Ghahramani, Z. (2016). *Dropout as a Bayesian approximation: Representing model uncertainty in deep learning.*
- Glasserman, P. & Yao, D.D. (1992). *Some guidelines and guarantees for common random numbers.*